Rolling most recent K mean, var, max, mode

Time Complexity

Space Complexity: O(# stations * k)

get_stations()	O(1)
update()	amortized O(1)
get()	O(1)
get_mode()	O(1)
get_max()	O(1)

```
from collections import deque
from math import sqrt

def get_sample_mean(total, count):
    return total / count

def get_sample_var(squared_total, total, count):
    if count == 1: return 0.0

    mean = get_sample_mean(total, count)
    squared_mean = get_sample_mean(squared_total, count)
    return count / (count - 1) * (squared_mean - mean * mean)

def get_sample_std(squared_total, total, count):
    var = get_sample_var(squared_total, total, count)
    if var < 0.0: return 0.0 # python's floating point arithmetic error
    return sqrt(var)

# buckets[count] -> set()
def add_to_mode(temp, val_freq_map, buckets, cur_max_count) # -> new max count
    old_valueCount = 0 if temp not in val_freq_map else val_freq_map[temp]
    new_valueCount = old_valueCount + 1

    val_freq_map[temp] = new_valueCount
```

```
    # Deal with old_valueCount
    if old_valueCount > 0:
        buckets[old_valueCount].remove(temp)
        if len(buckets[old_valueCount]) == 0: del buckets[old_valueCount]

    # Deal with new_valueCount
    if new_valueCount not in buckets: buckets[new_valueCount] = set()
    buckets[new_valueCount].add(temp)

    cur_max_count = max(cur_max_count, new_valueCount)
    return cur_max_count

def remove_from_mode(temp, val_freq_map, buckets, cur_max_count):
    old_valueCount = 0 if temp not in val_freq_map else val_freq_map[temp]
    if old_valueCount == 0: return cur_max_count
    new_valueCount = old_valueCount - 1

    # Deal with old_valueCount
    buckets[old_valueCount].remove(temp)
    if len(buckets[old_valueCount]) == 0: del buckets[old_valueCount]

    # Deal with new_valueCount
    if new_valueCount == 0: del val_freq_map[temp]
    else:
        val_freq_map[temp] = new_valueCount

        if new_valueCount not in buckets: buckets[new_valueCount] = set()
        buckets[new_valueCount].add(temp)

    if old_valueCount == cur_max_count:
        while cur_max_count > 0 and cur_max_count not in buckets:
            cur_max_count -= 1
    return cur_max_count

class RollingStatistics:
```

```
    def __init__(self, k):
        self.k = k
        self.timestamp = 0

        self.stations_map = {}            # station_name -> {"past temp": deque(), "stats": (total, squared_total)}
        self.stations_mode_map = {}  # station_name -> {“freq_map”: {}, “buckets”: {}, “max_count”: 0}
        self.stations_maxVal_map = {}       # station_name -> monoDesc_deque: [“val, timestamp”, ...]}

    def get_stations(self):
        return self.stations_map.keys()

    def update(self, station_name, temp):
        self.timestamp += 1

        # Init
        if station_name not in self.stations_map:
            self.stations_map[station_name] = {"past temp": deque(), "stats": (0.0, 0.0)}
            self.stations_mode_map[station_name] = {“freq_map”: {}, “buckets”: {}, “max_count”: 0}
            self.stations_maxVal_map[station_name] = deque()

        # Forward update
        total = self.stations_map[station_name]["stats"][0]
        squared_total = self.stations_map[station_name]["stats"][1]

        total += temp
        squared_total += temp ** 2
        self.stations_map[station_name]["past temp"].append(temp)

        new_max_count = add_to_mode(
            temp,
            self.stations_mode_map[station_name][“freq_map”],
            self.stations_mode_map[station_name][“buckets”],
```

```
                self.stations_mode_map[station_name][“max_count”]
            )

            while self.stations_maxVal_map[station_name] and
self.stations_maxVal_map[station_name][-1][0] < temp:
                self.stations_maxVal_map[station_name].pop()
            self.stations_maxVal_map[station_name].append( (temp, self.timestamp) )

            # Remove redundant
            if len(self.stations_map[station_name]["past temp"]) > self.k:
                removed_temp = self.stations_map[station_name]["past temp"].popleft()

                squared_total -= removed_temp ** 2
                total -= removed_temp

                new_max_count = remove_from_mode(
                    removed_temp,
                    self.stations_mode_map[station_name][“freq_map”],
                    self.stations_mode_map[station_name][“buckets”],
                    new_max_count
                )

            while (
                self.stations_maxVal_map[station_name] and
                self.stations_maxVal_map[station_name][0][1] <= self.timestamp - self.k
            ):
                self.stations_maxVal_map[station_name].popleft()

            self.stations_map[station_name]["stats"] = (total, squared_total)
            self.stations_mode_map[station_name][“max_count”] = new_max_count

        def get(self, station_name):
            if station_name not in self.stations_map: return (None, None)
            if len(self.stations_map[station_name]["past temp"]) < self.k: return (None, None)

            total = self.stations_map[station_name]["stats"][0]
```

```
        squared_total = self.stations_map[station_name]["stats"][1]
        count = len(self.stations_map[station_name][“past temp”])

        mean = get_sample_mean(total, count)
        std = get_sample_std(squared_total, total, count)

        return (round(mean, 3), round(std, 3))

    def get_mode(self, station_name):
        if station_name not in self.stations_map: return None
        if len(self.stations_map[station_name]["past temp"]) < self.k: return None

        max_count = self.stations_mode_map[station_name][“max_count”]
        buckets = self.stations_mode_map[station_name][“buckets”]
        return next(iter(buckets[max_count])) if max_count in buckets else None

    def get_max_val(self, station_name):
        if station_name not in self.stations_map: return None
        if len(self.stations_map[station_name]["past temp"]) < self.k: return None

        if len(self.stations_maxVal_map[station_name]) == 0: return None
        return self.stations_maxVal_map[station_name][0][0]
-
```